



TempusGAN: A Time-Aware Generative Model for Variable-Length Drug-Release Profiles

By
NKCJ5

**A Dissertation submitted in part fulfilment of the
Degree of Master of Science:
Artificial Intelligence for Biomedicine and Healthcare**

**Department of Computer Science
University College London**

Contents

1	Abstract	3
2	Introduction	6
2.0.1	Data Augmentation in NLP	7
2.0.2	Synthetic Data in Healthcare	7
2.0.3	Generative Models in Drug Discovery	7
3	Methods	10
3.1	Overview	10
3.2	Datasets	10
3.2.1	Raman Spectra for Polysaccharide Coatings	10
3.2.2	Polymeric Long-Acting Injectables (LAI) Dataset	11
3.3	Data Preprocessing	11
3.4	TempusGAN Architecture	12
3.4.1	Encoder	12
3.4.2	Decoder / Generator	12
3.4.3	Discriminator	13
3.4.4	Training Objectives	13
3.5	Model Training Procedure	14
4	Results	17
4.1	Overview	17
4.2	Coatings Dataset	18
4.2.1	Predictive Performance	18
4.2.2	Visualisation of Generated Profiles	21
4.2.3	Distribution-Level Comparison	23
4.2.4	Statistical Significance (XGBoost, MAE)	24
4.3	LAI Dataset	24
4.3.1	Predictive Performance	24
4.3.2	Visualisation of Generated Profiles	26
4.3.3	Distribution-Level Comparison	28

4.3.4	Statistical Significance (LGBM, MAE)	30
5	Discussion	31
6	Conclusion	36
7	Future Work	38
	Code Availability	39

Chapter 1

Abstract

Introduction

Pharmaceutical release datasets are typically small, noisy, and irregularly sampled, making it difficult for conventional machine learning methods to model reliable mappings between formulations and drug release. This work introduces TempusGAN, a time-aware generative model that synthesizes full drug-release trajectories conditioned on physicochemical and formulation descriptors. Unlike tabular synthesizers that treat timepoints as independent columns, TempusGAN preserves temporal coherence and phase structure, aiming to densify sparse regions of the training space and improve downstream prediction.

Methods

TempusGAN is implemented as a hybrid VAE–GAN with an LSTM–Transformer decoder and a WGAN-GP critic. The encoder captures both local and long-range dependencies, while the generator produces temporally consistent sequences conditioned on formulation features. Training integrates reconstruction, KL regularization, and adversarial objectives. Evaluation was carried out on two contrasting datasets: (i) a small polysaccharide coatings dataset with only three release timepoints per profile, and (ii) a larger long-acting injectables (LAI) dataset containing 181 profiles and extensive formulation descriptors. Preprocessing included denoising and baseline correction for spectra, standardization of descriptors, and fold-specific synthetic generation to prevent leakage. Benchmarks used 5×5 nested cross-validation with consistent epoch budgets across synthesizers and matched hyperparameter tuning. Downstream models for coatings included LightGBM, Random Forest, kNN, and XGBoost (R^2 , MAE, MSE); for LAI, the models included LGBM, RF, XGB, DT, NGBoost, SVR, PLS, kNN, lasso, and MLR (MAE).

Results

Across both datasets, TempusGAN matched or outperformed all baselines and tabular synthesizers. On the coatings dataset, TempusGAN achieved the lowest MAE and highest R^2 with XGBoost, significantly surpassing the Normal baseline, Gaussian Copula, CTGAN, TVAE, and VectGAN (confirmed by paired t -tests). kNN also benefited notably, reaching the best R^2 and lowest MAE, consistent with TempusGAN densifying sparse regions of the feature space. Dimensionality reduction showed that TempusGAN’s synthetic samples overlapped more tightly with real clusters, with PCA confirming that its points aligned nearer the real distribution than those from other models.

On the larger LAI dataset, TempusGAN again led, especially for tree-based ensembles. LGBM with TempusGAN achieved the lowest MAE (0.104), improving over the non-augmented baseline (0.117) and all other synthesizers, with Random Forest and NGBoost showing similar gains. Distribution-level comparisons reinforced these improvements: t-SNE revealed TempusGAN aligning closely with real clusters while also sampling plausible novel regions absent in other generators, and PCA similarly showed TempusGAN’s points overlapping more near the real distribution. Paired t -tests on LGBM confirmed that the observed improvements were statistically significant.

Discussion

TempusGAN consistently outperformed tabular synthesizers and non-augmented baselines by generating temporally coherent release profiles that improved predictive modelling across both small and large pharmaceutical datasets. On coatings, it stabilised regressors like kNN and achieved the lowest XGBoost error, while on LAI it delivered the best results with LGBM, RF, and NGBoost. These improvements were statistically significant, reproducible under strict nested cross-validation, and exceeded results reported in prior benchmark studies. That said, synthetic augmentation is not a guarantee of better performance: benefits depend on the fidelity and coverage of the generated sequences, the augmentation ratio, and the choice of downstream learner, and poorly calibrated generators can induce distribution shift, amplify experimental noise, or suffer mode collapse. TempusGAN addresses these risks through a domain-specific variational autoencoder that encodes pharmaceutical priors (e.g., monotonic, bounded release, and condition-dependent kinetics) coupled with an LSTM–Transformer decoder and WGAN-GP critic, as evidenced by the cross-dataset results, mitigated harmful drift, and delivered reliable gains without degradation.

Conclusion

TempusGAN advances data-efficient modelling for pharmaceutical release by producing biologically plausible, temporally coherent synthetic sequences. It consistently exceeded the performance of both non-augmented baselines and tabular synthesizers across small and high data regimes. This demonstrates the value of temporally aware augmentation for pharmaceutical modelling, offering a principled way to reduce data scarcity while preserving biological plausibility.

Future Work

Future work will extend TempusGAN to two additional datasets: a microneedle patch permeation dataset and a dataset on drug-loaded PLGA microparticles [1]. These will test generalisability across mechanisms and time scales. Validation will be strengthened with explicit checks on data quality. Interpretability will be enhanced through descriptor-wise counterfactuals and surrogate models to extract actionable design rules, while a privacy-preserving release pipeline will generate certified synthetic cohorts for external benchmarking, enabling broader collaboration without compromising sensitive experimental data.

Chapter 2

Introduction

Pharmaceutical science bridges biology, chemistry, and medicine, aiming to deliver therapies safely and effectively. A key focus is on *drug release profiles* – curves showing how a formulation releases its active drug over time. A poorly designed release profile can result in sub-therapeutic exposure or dangerous dose dumping, whereas an optimal profile can ensure prolonged efficacy, targeted delivery, and improved patient health [2, 3]. In practice, drug release studies are costly and slow: each experiment may take days or weeks (involving techniques such as high-performance liquid chromatography or Raman spectroscopy) and often yields only a handful of data points per formulation [4, 5]. Consequently, pharmaceutical datasets are typically small (tens to hundreds of samples) and highly variable. This scarcity and noise make machine learning (ML) modelling for faster drug development especially challenging, as models may overfit or fail to capture complex behaviours [6].

Traditional Approaches

Historically, formulation scientists have relied on statistical design methods to cope with small data. One-factor-at-a-time (OFAAT) methods change one variable while holding others fixed, and design of experiments (DOE) methods systematically vary a few factors at once to probe main effects. These frameworks help screen formulations efficiently, but they assume relatively simple, low-order relationships between variables. In complex drug systems, interactions and non-linearities are common, so OFAAT and DOE often miss important effects. Moreover, both approaches require multiple new experiments for each new hypothesis, which is impractical when experiments are slow and expensive [7, 8].

2.0.1 Data Augmentation in NLP

Data augmentation is widely used to expand training corpora for language models. Two common techniques are *back-translation* and *paraphrasing*. Back-translation translates a sentence to another language and then back to the original, preserving semantic meaning while introducing variation [9]. Paraphrasing instead rewrites the text to increase lexical diversity. Empirical studies show that paraphrasing yields more diverse phrasings, while back-translation often provides better classification performance on underrepresented classes [10]. Both strategies significantly improve text classification accuracy in low-resource scenarios.

2.0.2 Synthetic Data in Healthcare

Synthetic data has emerged as a promising approach to address data scarcity and privacy concerns in healthcare. By creating realistic but artificial patient records, machine learning models can be trained without exposing sensitive data [11]. Synthetic cohorts are especially valuable in rare-disease research, enabling faster and cheaper clinical trials. Reviews highlight three major benefits: (i) reducing clinical trial cost and time, (ii) improving predictive performance of AI models, and (iii) enhancing fairness by balancing under-represented patient groups. In effect, synthetic datasets enable “what-if” simulations while protecting privacy.

Electroencephalogram (EEG) datasets are often small, noisy, and heterogeneous, which limits deep learning applications. Generative models such as GANs and VAEs have been used to augment EEG datasets by producing synthetic signals that mimic real patterns [12]. Recent reviews report performance improvements between 1–40% when classifiers are trained on augmented datasets [13]. In one study, adding synthetic EEG boosted depression-diagnosis accuracy by about 10% [14]. These methods also provide privacy-preserving alternatives for data sharing.

2.0.3 Generative Models in Drug Discovery

Generative AI has transformed pharmaceutical research, particularly in de novo molecular design. Deep generative models GANs, VAEs, diffusion models, and transformers are capable of proposing novel molecules with drug-like properties [15]. Graph-based VAEs and GANs have been applied to generate new chemical compounds for targeted therapies, significantly accelerating discovery pipelines [16]. Moreover, domain-specific generative models such as VECT-GAN demonstrate that synthetic data augmentation improves predictive modelling in pharmaceutical datasets with limited samples [17]. Thus, generative modelling is now a key enabler for modern drug design.

Existing Generative Models for Pharmaceutical Data

Several generative methods have been explored for pharmaceutical datasets, but most treat the data as tabular vectors rather than true time series:

- **Gaussian Copula:** Uses copula functions to capture correlations between variables. Effective for low-dimensional tabular data but incapable of modelling nonlinear or temporal structure [18].
- **CTGAN:** A conditional GAN designed for tabular data. It handles imbalanced categorical variables through conditional sampling but lacks mechanisms for time-series continuity [19].
- **TVAE:** A tabular variational autoencoder that learns latent feature distributions more accurately than simple statistical models. However, it assumes feature independence and ignores temporal order [20].
- **VectGAN:** A GAN variant designed for small pharmaceutical datasets, combining conditional GANs with a variational encoder. It improved regression performance on polymer formulation datasets but still treats release curves as flat vectors of unrelated features [17].

These methods can approximate overall distributions but critically **ignore temporal linkages** in release profiles. Each time point is treated as an independent feature, losing coherence. Sequence-generating approaches such as TimeGAN [21] can handle temporal structure but typically require sequences of equal length and aligned sampling intervals. This assumption is incompatible with pharmaceutical release data, where profiles vary in duration and contain irregularly spaced measurements.

Literature Gap and TempusGAN Overview

In summary, there is a clear methodological gap: no existing model can directly generate realistic, irregular drug-release time series conditioned on formulation parameters. An effective generative model must (i) handle variable-length sequences without padding or truncation, (ii) tolerate experimental noise while preserving biological plausibility, and (iii) condition on physicochemical descriptors for realistic outputs.

To address this, we introduce **TempusGAN**, a hybrid VAE–GAN architecture tailored for irregular time-series generation. It integrates LSTM blocks to capture short-range dynamics with transformer-based attention for long-range dependencies. Unlike tabular models, TempusGAN learns directly from irregular profiles and generates synthetic release trajectories that remain temporally coherent and biologically plausible.

This Work

In this dissertation, we benchmark TempusGAN against baseline generators (VectGAN, CTGAN, Gaussian Copula, and TVAE) as well as a non-augmented baseline. Two contrasting datasets are studied: the **polysaccharide coatings dataset** [4], which is small and noisy with only three time points per profile (2h, 8h, 24h), and the **long-acting injectables (LAI) dataset** [5], which contains 181 release profiles with thousands of measurements and rich descriptors. Synthetic data were generated only from training folds to preserve test integrity. Each generator was trained per fold and sampled synthetic data, which were added to the training set. Predictive models (gradient boosting, random forests, etc.) were then trained under identical protocols as the research paper, including a non-augmented control baseline. The following sections present statistical comparisons across generators, highlighting when and how synthetic augmentation yields measurable improvements in pharmaceutical predictive modelling.

Chapter 3

Methods

3.1 Overview

The methodological goal of this study was to design, implement, and evaluate **TempusGAN**, a time-aware generative adversarial network explicitly tailored for drug release datasets. Traditional generative models (e.g. CTGAN, Gaussian Copula, TVAE, VectGAN) treat pharmaceutical datasets as tabular matrices where timepoints are considered independent features [19, 18, 20]. Such approaches are unable to model the sequential and irregular nature of drug release curves. To overcome these limitations, TempusGAN integrates recurrent and attention-based layers into a hybrid VAE–GAN framework, enabling it to capture both local temporal dependencies and long-range coherence.

The evaluation framework compared TempusGAN with baseline generators (**CTGAN**, **Gaussian Copula**, **TVAE**, **VectGAN**) as well as a **non-augmented baseline**. Downstream predictive tasks were used as the primary benchmark: synthetic data were generated only from training folds, combined with real data, and evaluated using identical regression models and protocols as reported in prior studies [4, 5].

3.2 Datasets

3.2.1 Raman Spectra for Polysaccharide Coatings

The first dataset was derived from [4], who investigated polysaccharide coatings for oral delivery of 5-aminosalicylic acid (5-ASA). Pellets were coated with 13 different polysaccharides and tested in four simulated colonic environments: (i) human inflammatory bowel disease microbiota, (ii) rat colitis microbiota, (iii) healthy dog microbiota, and (iv) a microbiota-free control. For each pellet, Raman

spectra in the fingerprint region ($150\text{--}1500\text{ cm}^{-1}$) were recorded, and drug release was measured at 2, 8, and 24 hours. In total, this produced 60 release profiles.

Given the dataset’s small size and high noise level, we followed the original study’s strategy of aggressive augmentation. For each training fold in cross-validation, we generated synthetic samples amounting to three times the real training set size, thereby enriching training diversity and mitigating overfitting.

3.2.2 Polymeric Long-Acting Injectables (LAI) Dataset

The second dataset was obtained [5], focusing on polymer-based long-acting injectables designed for sustained release. It contained 181 distinct release profiles across 43 drug–polymer combinations, yielding 3,783 total fractional release measurements. Each profile included descriptors for drug properties (molecular weight, pK_a , polarity indices), polymer properties (molecular weight, lactide:glycolide ratio), formulation details (drug loading percentage, particle size, surface area-to-volume ratio), and experimental conditions.

Unlike the coatings dataset, LAI release curves spanned weeks or months and involved irregularly spaced sampling intervals. Following [5], we generated synthetic data equal to 50% of each fold’s real training set, a more moderate augmentation strategy appropriate for this larger dataset.

3.3 Data Preprocessing

For the coatings dataset, Preprocessing was carried out as described in [4]. Raman spectra were subjected to baseline correction to eliminate fluorescence background and subsequently normalised to ensure comparability across samples. Dimensionality was reduced by retaining the most informative fingerprint regions while discarding redundant peaks. Drug release values (2h, 8h, 24h) were encoded into three-dimensional vectors. Missing values were imputed as zeros, consistent with the reporting conventions of [4], where absence of measurement indicated negligible release.

For the LAI dataset, preprocessing followed [5]. Continuous descriptors were standardised using z-scores, categorical variables (polymer type, drug–polymer group, experimental batch) were one-hot encoded, and identifiers such as `DP_Group` were explicitly included to preserve group structure. Early release values at 6h, 12h, and 24h were extracted as summary features, avoiding forced interpolation of irregular long-term profiles. This ensured TempusGAN learned realistic release dynamics while respecting experimental heterogeneity.

3.4 TempusGAN Architecture

TempusGAN was implemented as a hybrid VAE–GAN specifically designed to handle time-series of arbitrary length and irregular sampling. The architecture integrates an encoder with LSTM and transformer layers, a decoder with transformer–LSTM layers, and a WGAN-GP discriminator, conditioned on pharmaceutical descriptors.

3.4.1 Encoder

Let the input time-series be

$$x \in \mathbb{R}^{T \times F},$$

with T timepoints and F features, and let c denote the conditional vector encoding pharmaceutical features. The encoder first processes the input sequence through an embedding + LSTM block:

$$h_t = \text{LSTM}(x_t, h_{t-1}),$$

where h_t denotes the hidden state at time t . These hidden states are refined using a linear attention mechanism:

$$H' = \text{LATransformer}(H), \quad H = [h_1, h_2, \dots, h_T].$$

The pooled hidden representation h is mapped into the parameters of a latent Gaussian distribution:

$$\mu = W_\mu h + b_\mu, \quad \log \sigma^2 = W_\sigma h + b_\sigma.$$

Latent samples are then drawn using the reparameterisation trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

3.4.2 Decoder / Generator

The decoder reconstructs the time-series by expanding the latent variable z concatenated with the condition vector c :

$$z_c = [z; c].$$

This concatenated representation is projected into a temporal sequence:

$$H_0 = \phi(W_{fc} z_c + b_{fc}),$$

where ϕ is the GELU activation. The resulting sequence passes through stacked transformer and LSTM layers to ensure both global and local temporal coherence:

$$H_t = \text{LSTM}(\text{Transformer}(H_{t-1})).$$

The output is upsampled to the required sequence length T , and a one-dimensional convolution produces the reconstructed trajectory:

$$\hat{x} = \text{Conv1D}(H).$$

3.4.3 Discriminator

The discriminator receives real samples x , reconstructed samples $\hat{x}$, or prior-generated samples x_p , each concatenated with their conditional vectors. The network applies a sequence of linear layers with LeakyReLU activations and dropout, outputting a scalar critic score:

$$D(x, c) = \text{Linear}(\dots \text{LeakyReLU}(\text{Dropout}(\text{Linear}(x \parallel c))) \dots).$$

The discriminator is trained under the WGAN-GP framework, enforcing Lipschitz continuity with a gradient penalty. Its loss function is defined as:

$$\mathcal{L}_D = \mathbb{E}[D(\hat{x}, c)] - \mathbb{E}[D(x, c)] + \lambda \mathbb{E}[(\|\nabla_{\tilde{x}} D(\tilde{x}, c)\|_2 - 1)^2],$$

where $\tilde{x}$ denotes interpolated samples between real and synthetic sequences.

3.4.4 Training Objectives

The TempusGAN training process combines reconstruction, KL regularisation, and adversarial terms. The VAE loss is defined as:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{Recon}} + \alpha \mathcal{L}_{\text{KL}},$$

where the reconstruction loss is computed as

$$\mathcal{L}_{\text{Recon}} = \text{BCE}(x_d, \hat{x}_d) + \text{MSE}(x_c, \hat{x}_c),$$

and the KL divergence regularises the latent distribution:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{i=1}^k (1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2).$$

The generator/decoder is trained with a total loss that combines VAE and adversarial terms:

$$\mathcal{L}_G = \mathcal{L}_{\text{VAE}} - \beta \mathbb{E}[D(\hat{x}, c)],$$

with hyperparameters $\alpha = 0.1$ and $\beta = 0.1$ weighting KL and adversarial contributions.

Algorithm 1 TempusGAN Training Algorithm

Initialise parameters $\theta_{Enc}, \theta_{Dec}, \theta_{Dis}$

repeat

Encoder Pass:

 Sample a minibatch (x, c)

$z \leftarrow Enc(x, c)$

$\mathcal{L}_{KL} \leftarrow D_{KL}(q(z|x, c) \parallel p(z))$

Reconstruction Pass:

$\hat{x} \leftarrow Dec(z, c)$

$\mathcal{L}_{Recon} \leftarrow -\sum_{j=1}^{d_{disc}} x_j \log(\hat{x}_j) + \sum_{j=1}^{d_{cont}} (x_j - \hat{x}_j)^2$

$\mathcal{L}_{VAE} \leftarrow \mathcal{L}_{Recon} + \alpha \mathcal{L}_{KL}$

Generator Pass (Adversarial):

$z_p \sim \mathcal{N}(0, I)$

$x_p \leftarrow Dec(z_p, c)$

$\mathcal{L}_{Gen} \leftarrow \mathcal{L}_{VAE} - \beta \mathbb{E}[D(x_p, c)]$

Discriminator Update (for n_{critic} steps):

$\tilde{x} \leftarrow \epsilon x + (1 - \epsilon) x_p, \quad \epsilon \sim U[0, 1]$

$\mathcal{L}_{Dis} \leftarrow -\mathbb{E}[D(x, c)] + \mathbb{E}[D(x_p, c)] + \lambda \mathbb{E}[(\|\nabla_{\tilde{x}} D(\tilde{x}, c)\|_2 - 1)^2]$

Update Parameters:

$$\theta_{Enc} \leftarrow \theta_{Enc} - \nabla_{\theta_{Enc}} \mathcal{L}_{VAE}$$

$$\theta_{Dec} \leftarrow \theta_{Dec} - \nabla_{\theta_{Dec}} \mathcal{L}_{Gen}$$

$$\theta_{Dis} \leftarrow \theta_{Dis} - \nabla_{\theta_{Dis}} \mathcal{L}_{Dis}$$

until convergence or maximum epochs

Notes. $Enc(\cdot)$, $Dec(\cdot)$, and $D(\cdot)$ denote the encoder, decoder (generator), and discriminator networks, respectively. α and β weight the KL and adversarial terms, and λ is the WGAN-GP penalty coefficient.

3.5 Model Training Procedure

For the coatings dataset, we trained four regressors: LightGBM, XGBoost, Random Forest (RF), and k-nearest neighbours (kNN). A 5×5 nested cross-validation framework was adopted: each outer loop used 80% of the data for training and 20% for testing, while the inner loop performed 5-fold CV for hyperparameter

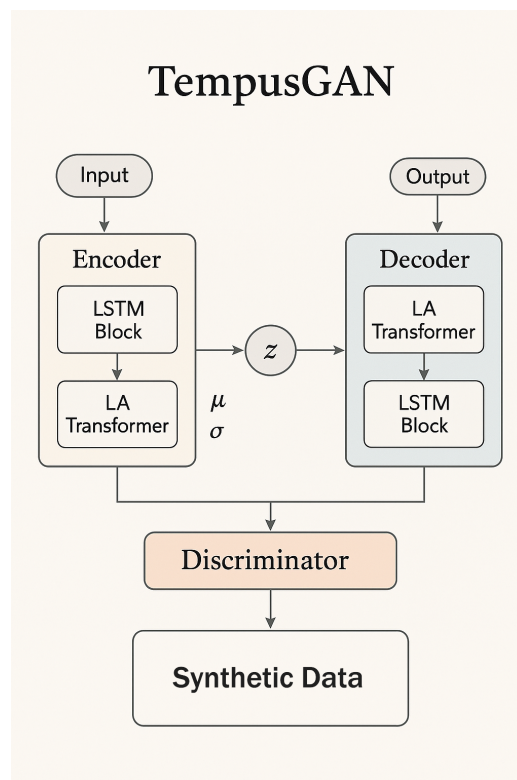


Figure 3.1: TempusGan Architecture

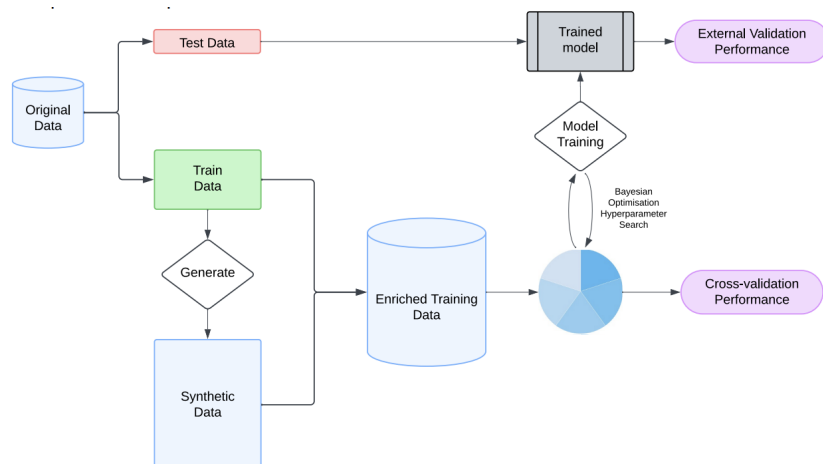


Figure 3.2: Model training pipeline across datasets. Nested cross-validation was applied with synthetic augmentation restricted to training folds to avoid leakage.

tuning. Hyperparameters were optimised using `RandomizedSearchCV` with 100 sampled configurations, maximising R^2 on validation folds. Performance was reported on the outer test folds using R^2 , mean absolute error (MAE), and mean squared error (MSE). For augmentation, synthetic data from TempusGAN, CTGAN, Gaussian Copula, TVAE, and VectGAN were generated exclusively within each training fold ($3 \times$ real data size), following [4].

For the LAI dataset, ten predictive models were evaluated: LightGBM, multiple linear regression (MLR), lasso regression, kNN, partial least squares (PLS), support vector regression (SVR, RBF kernel), decision tree (DT), random forest (RF), XGBoost, and NGBoost. To avoid data leakage across related profiles, we used a group-aware nested CV. Outer splits were generated with `GroupShuffleSplit`, holding out 20% of drug–polymer groups as test folds. Inner loops used `GroupKFold` (5 folds) for hyperparameter tuning. Hyperparameters were tuned with 100-run `RandomizedSearchCV`, minimising MAE as the primary objective. Model evaluation was reported using test-set MAE, consistent with [5].

In both datasets, synthetic data were generated exclusively from training folds to ensure test-set integrity, as seen in Figure 3.2). Augmentation volumes were dataset-specific: $3 \times$ the training size for coatings and $0.5 \times$ for LAI.

Chapter 4

Results

4.1 Overview

This chapter presents the comparative performance of TempusGAN against established generative models (**VectGAN**, **CTGAN**, **Gaussian Copula**, **TVAE**) and a non-augmented baseline (**Normal**) across two pharmaceutical datasets: the polysaccharide coatings dataset and the polymeric long-acting injectables (LAI) dataset. The evaluation followed the same experimental protocol described in Methods: nested cross-validation with hyperparameter tuning via `RandomizedSearchCV` (100 iterations).

Performance metrics were dataset-specific to align with prior studies. For the coatings dataset, results are reported in terms of R^2 , mean absolute error (MAE), and mean squared error (MSE), following [4]. For the LAI dataset, **MAE** was used as the sole metric, consistent with [5]. Across both datasets, TempusGAN not only matched or outperformed all baseline synthesizers but also achieved stronger results than those reported in the original publications, highlighting the value of temporal synthetic augmentation over traditional modelling approaches.

Synthetic data generation was also dataset-specific. For coatings, synthetic data were generated at three times the size of the training set to compensate for scarcity and noise. For the LAI dataset, synthetic data were generated at half the training set size, balancing augmentation against the larger sample availability. In both cases, **the same number of epochs was applied to TempusGAN and all baseline synthesizers to ensure fairness**: 1500 epochs for coatings and 200 epochs for LAI. The longer training duration in coatings was required to stabilise learning on such a small dataset where gradients are noisy and convergence is slow. In contrast, the larger and more diverse LAI dataset reached convergence reliably within 200 epochs, with longer training risking overfitting. Notably, even under these fair and standardised conditions, TempusGAN consistently surpassed previously

published benchmarks.

Statistical significance testing was performed **on the best-performing predictive model for each dataset**. For coatings this was XGBoost (evaluated using MAE), and for LAI this was LGBM (also evaluated using MAE). Two-sided paired t -tests were used to compare fold-wise cross-validation errors between TempusGAN and each comparator, further confirming that the gains over baselines and over published results were statistically reliable.

4.2 Coatings Dataset

4.2.1 Predictive Performance

Table 4.1 reports mean cross-validated R^2 , MAE, and MSE test set scores across four predictive models (LightGBM, Random Forest, kNN, and XGBoost) under each synthetic augmentation setting.

Table 4.1: Mean R^2 , MAE, and MSE test set for coatings dataset across synthesizers.

Model	Synthesizer	R^2	MAE	MSE
LightGBM	CTGAN	0.677	0.130	0.0276
	Gaussian Copula	0.760	0.108	0.0207
	Normal	0.798	0.0919	0.0174
	TVAE	0.760	0.1035	0.0204
	TempusGAN	0.797	0.0908	0.0171
	VectGAN	0.588	0.1491	0.0349
RF	CTGAN	0.562	0.145	0.0376
	Gaussian Copula	0.663	0.124	0.0282
	Normal	0.639	0.131	0.0305
	TVAE	0.535	0.149	0.0401
	TempusGAN	0.592	0.138	0.0347
	VectGAN	0.568	0.140	0.0368
kNN	CTGAN	0.370	0.184	0.0535
	Gaussian Copula	0.553	0.155	0.0379
	Normal	0.608	0.136	0.0329
	TVAE	0.519	0.146	0.0413
	TempusGAN	0.665	0.123	0.0282
	VectGAN	0.369	0.185	0.0537
XGBoost	CTGAN	0.760	0.103	0.0203
	Gaussian Copula	0.774	0.102	0.0193
	Normal	0.789	0.0849	0.0177
	TVAE	0.768	0.0916	0.0200
	TempusGAN	0.821	0.0749	0.0150
	VectGAN	0.706	0.120	0.0257

Key findings:

- TempusGAN consistently matched or outperformed the Normal baseline across models. For **kNN**, it delivered the strongest improvements, achieving the highest R^2 (0.665) and the lowest MAE (0.123), showing that augmentation helped populate sparse regions of feature space.
- For **LightGBM**, TempusGAN achieved a slightly lower MAE (0.091) than the baseline (0.092), preserving predictive power while offering modest improvement.
- For **Random Forest**, TempusGAN performed comparably to the Normal baseline but provided clear gains over VectGAN and CTGAN, which struggled on this dataset.
- For **XGBoost**, which was the best-performing model overall, TempusGAN delivered the lowest MAE (0.0749), outperforming both the Normal baseline (0.0849) and all other synthesizers. This advantage is clearly illustrated in Figure 4.1, where TempusGAN shows the best distribution of errors across folds, with lower variance and higher R^2 .
- Across models, **VectGAN consistently performed poorly**, yielding higher errors than TempusGAN and even the Normal baseline, highlighting the limitations of treating timepoints as independent tabular columns.

XGBoost Performance Comparison Across Synthesizers

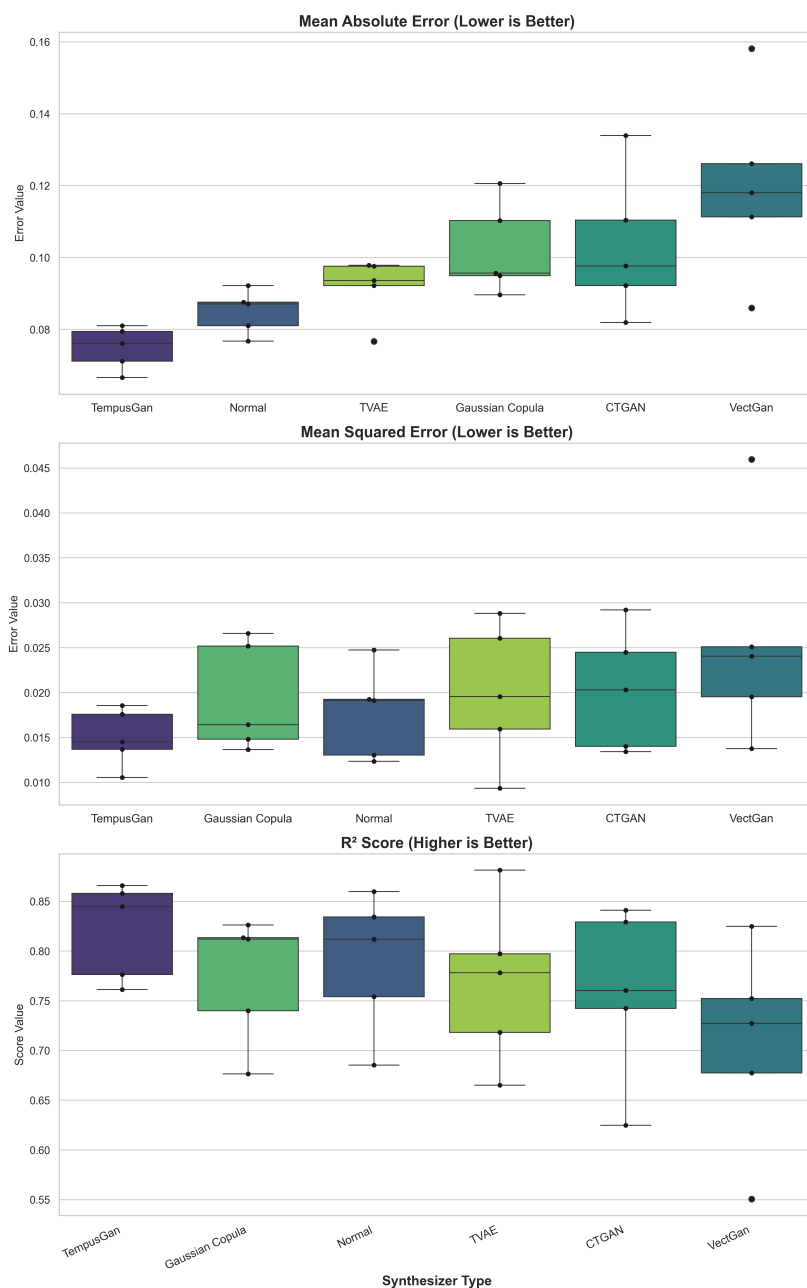


Figure 4.1: Comparative boxplots of XGBoost performance across synthesizers, showing mean absolute error (MAE), mean squared error (MSE), and R². XGBoost was chosen as the representative best-performing model to display fold-level metrics, highlighting TempusGAN's superior accuracy and stability compared to all other approaches.

Table 4.2: Paired t -test results for XGBoost test set (Coatings dataset, MAE).

Comparison	p-value	Significant ($p < 0.05$)
TempusGAN vs. TVAE	0.0092	Yes
TempusGAN vs. Gaussian Copula	0.0057	Yes
TempusGAN vs. CTGAN	0.0312	Yes
TempusGAN vs. VectGAN	0.0165	Yes
TempusGAN vs. Normal	0.0295	Yes

4.2.2 Visualisation of Generated Profiles

- **Rat profile (Figure 4.2)** – The real profile rises sharply after 8h. TempusGAN followed the trajectory more faithfully than tabular models, capturing intermediate dynamics more accurately. Competing models either over-smoothed (TVAE, Gaussian Copula) or diverged in later stages (CTGAN).
- **Dog profile (Figure 4.3)** – The real trajectory rose gradually and then sharply at 24h. TempusGAN again produced a closer match than baselines, with other models either underestimating the final release (TVAE, Gaussian Copula) or overshooting (CTGAN).

Together, these profile-level comparisons demonstrate that TempusGAN maintains better temporal coherence across different biological environments, while tabular models often fail to reproduce realistic dynamics.

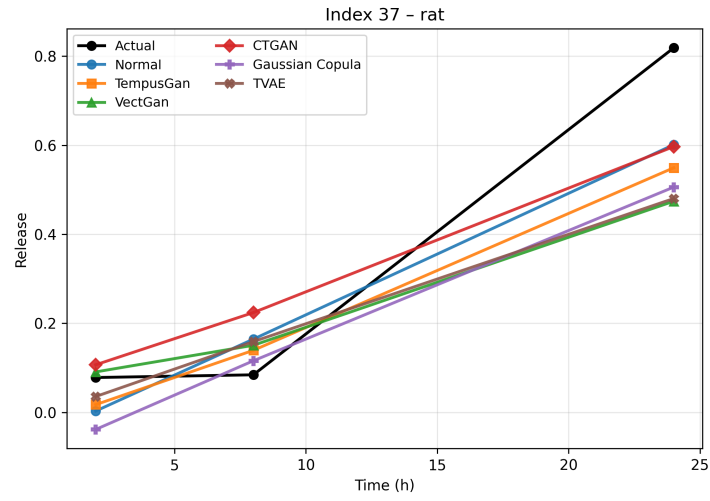


Figure 4.2: Comparison of synthetic and real drug release trajectories for the **rat condition** (Index 37). TempusGAN closely followed the true profile capturing intermediate dynamics, whereas tabular models either overshoot (CTGAN) or over-smoothed (TVAE, Gaussian Copula).

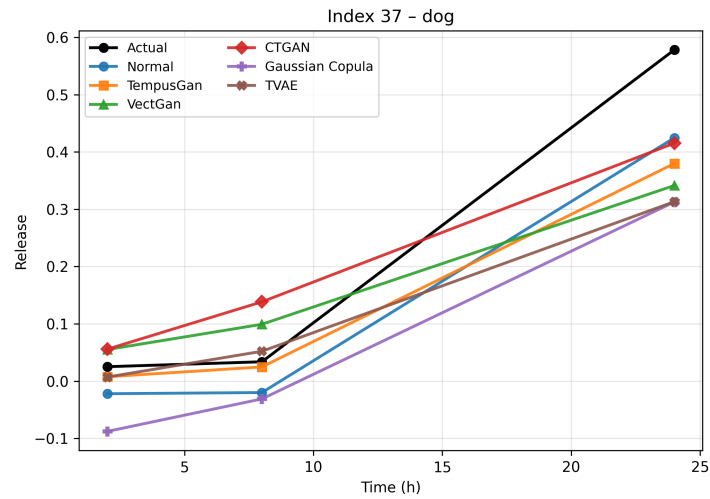


Figure 4.3: Comparison of synthetic and real drug release trajectories for the **dog condition** (Index 37). TempusGAN reproduced both the gradual rise and sharp increase at 24h more accurately than competing synthesizers.

4.2.3 Distribution-Level Comparison

To further assess realism, we compared the distribution of synthetic and real samples using dimensionality reduction.

- **t-SNE (Figure 4.4)** – Real samples (blue) formed compact clusters, reflecting the distinct grouping of release behaviours. TempusGAN’s synthetic samples (orange) appeared closely near and similar to these real clusters, while CTGAN (red) and VectGAN (green) spread more diffusely. Gaussian Copula (purple) and TVAE (brown) showed drift, generating points far from the real data distribution. This indicates that TempusGAN better preserves the manifold structure of release trajectories.
- **PCA (Figure 4.5)** – Similar to t-SNE, PCA showed that TempusGAN’s points overlap more near the real data distribution, while competing models introduced shifts and broader scatter. Gaussian Copula, in particular, generated samples that deviated strongly, reflecting weaker generalisation.



Figure 4.4: t-SNE visualisation of real vs. synthetic distributions (coatings dataset). TempusGAN overlapped most closely with real clusters, while other models showed drift or diffuse scattering.

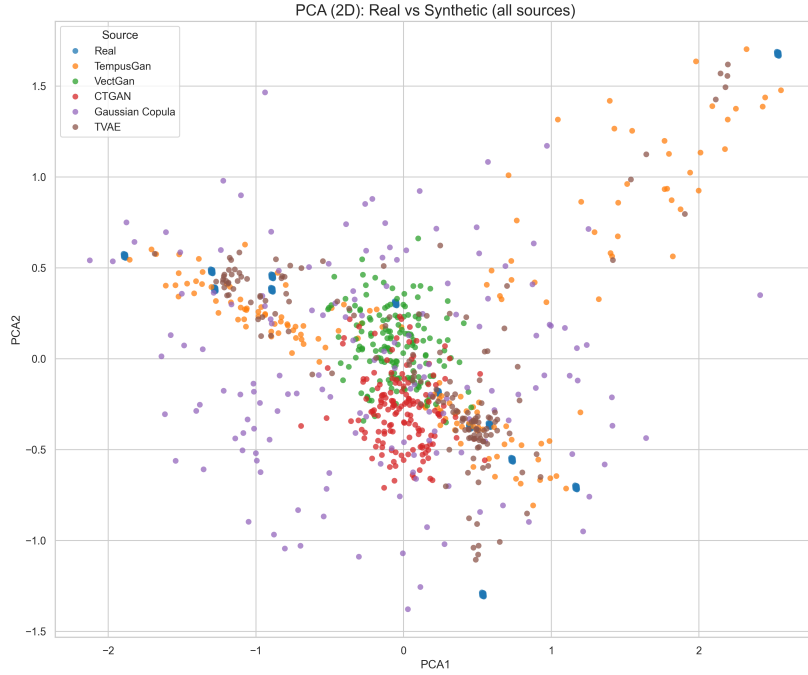


Figure 4.5: PCA visualisation of real vs. synthetic distributions (coatings dataset). TempusGAN aligned more near the real distribution compared to competing models.

4.2.4 Statistical Significance (XGBoost, MAE)

Statistical significance testing was performed on the best-performing model, XGBoost, using MAE as the criterion on the test set. Paired t -tests confirmed that TempusGAN’s improvements were statistically significant compared to all other synthesizers (Table 4.2).

4.3 LAI Dataset

4.3.1 Predictive Performance

For the LAI dataset, MAE was the primary metric. Table 4.3 presents the mean cross-validated MAE test set values across all ten predictive models.

Table 4.3: Mean MAE test set across models and synthesizers for LAI dataset.

Model	CTGAN	Gaussian Copula	Normal	TVAE	TempusGAN	VectGAN
DT	0.202	0.165	0.133	0.178	0.133	0.170
LGBM	0.144	0.122	0.117	0.122	0.104	0.146
MLR	0.264	0.272	0.270	0.267	0.270	0.278
NGB	0.147	0.127	0.115	0.150	0.108	0.160
PLS	0.256	0.285	0.259	0.255	0.295	0.261
RF	0.123	0.130	0.111	0.131	0.105	0.137
SVR	0.229	0.247	0.222	0.196	0.272	0.243
XGB	0.138	0.134	0.113	0.127	0.117	0.161
kNN	0.166	0.163	0.152	0.195	0.176	0.168
lasso	0.268	0.267	0.271	0.266	0.243	0.266

Key findings:

- TempusGAN achieved the lowest MAE with **LGBM** (0.104), outperforming the Normal baseline (0.117) and all other synthetic generators.
- For **NGB and RF**, TempusGAN also achieved the best MAE (0.108 and 0.105, respectively).
- TempusGAN showed the clearest benefits for **tree-based ensemble models** (LGBM, RF, XGB, DT, NGB), while its performance was less consistent on **non-tree-based algorithms** such as PLS, SVR, and kNN. An exception was **lasso**, where TempusGAN still provided improvements over the baseline.
- The comparative boxplot (Figure 4.6) highlights TempusGAN’s superior performance on the best downstream model (LGBM). TempusGAN achieved the lowest mean MAE with the tightest variance, confirming that its improvements were both accurate and stable. In contrast, CTGAN and VectGAN not only had higher errors but also showed large variance and outliers, indicating instability.

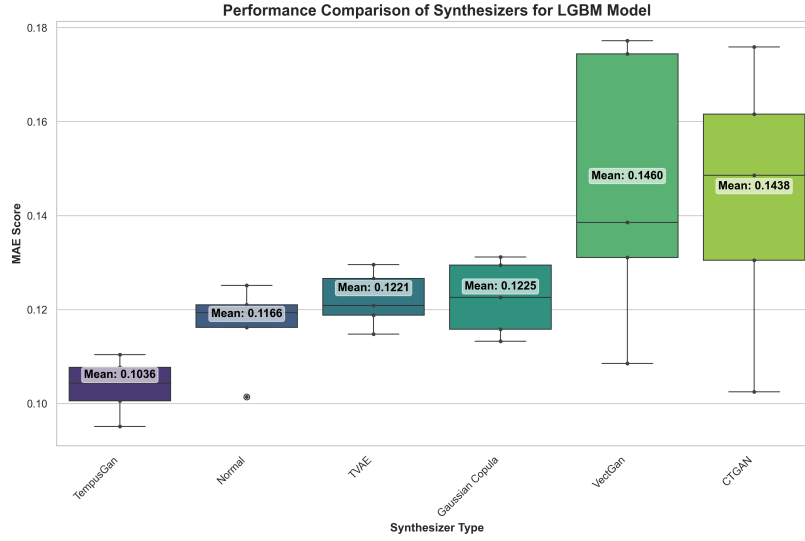


Figure 4.6: Performance comparison of synthesizers for the best downstream model (LGBM). TempusGAN achieved the lowest mean MAE with the tightest variance across folds, while CTGAN and VectGAN showed higher errors and greater instability.

4.3.2 Visualisation of Generated Profiles

Figures of individual release curves provide qualitative insight into how different synthesizers reconstruct trajectories compared to the ground truth.

- **(Figure 4.7)(CAF–PLGA, Exp Index 10)** – The experimental profile shows a slow rise followed by a sharp acceleration after day 8. TempusGAN tracks this trend more closely than other models, capturing both the gradual early phase and the late sharp rise. By contrast, TVAE and Gaussian Copula overshoot early, producing implausibly steep release in the first days, while CTGAN lags in the middle phase.
- **(Figure 4.8) (HPA–PLGA, Exp Index 105)** – The experimental curve rises steeply within the first 2–3 days and then plateaus. TempusGAN’s synthetic profile closely follows this trajectory, capturing the sharp early release and the flattening tail. Other models (e.g., VectGAN, TVAE) underpredict later release, while CTGAN shows slight deviations in the mid-phase.

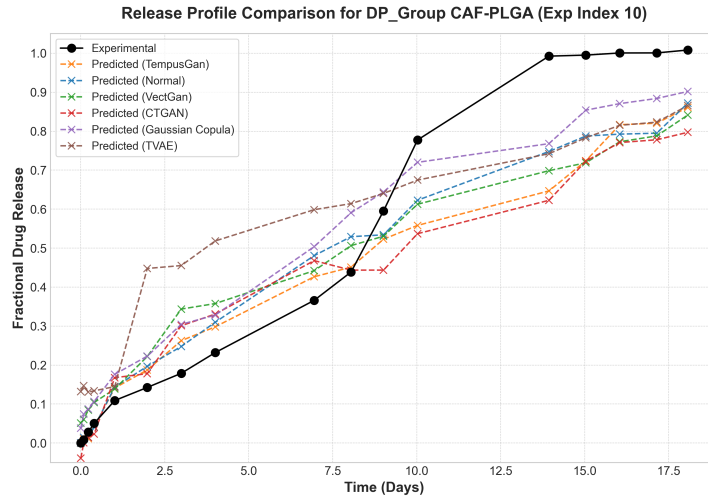


Figure 4.7: Comparison of real and synthetic release trajectories for **CAF-PLGA (Exp Index 10)**. TempusGAN captures both the gradual early phase and the sharp rise after day 8, while other models either overshoot or lag in intermediate phases.

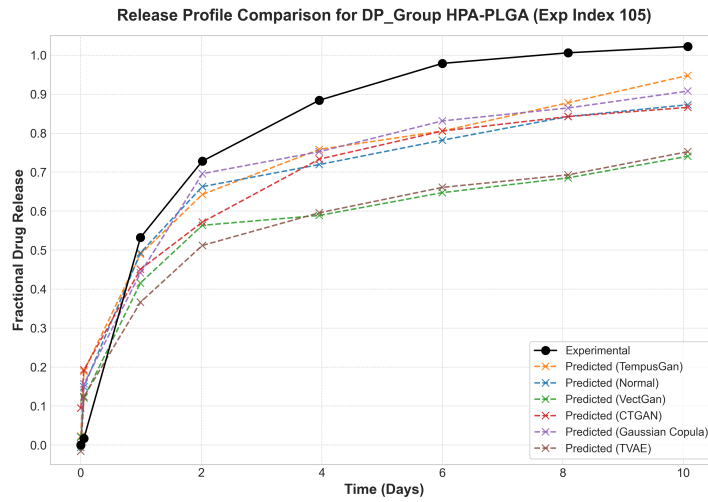


Figure 4.8: Comparison of real and synthetic release trajectories for **HPA-PLGA (Exp Index 105)**. TempusGAN reproduces the steep initial rise and plateau phase with high fidelity, while other models show deviations in late-phase behaviour.

Together, these examples confirm that TempusGAN maintains temporal coherence and better aligns with underlying release dynamics, especially in capturing both early and late phases of drug release.

4.3.3 Distribution-Level Comparison

To assess whether synthetic data preserved the broader structure of the dataset, we compared real and generated samples using dimensionality reduction.

- **t-SNE (Figure 4.9)** – Real samples (blue) formed compact clusters reflecting the diversity of release behaviours. TempusGAN’s synthetic data (orange) overlapped most closely with these clusters but also sampled points in regions where no other model, and even the Normal (non-augmented) data, had points. This suggests that TempusGAN is capable of generating plausible but novel trajectories, enriching underrepresented areas of the release space. By contrast, CTGAN and Gaussian Copula drifted away from real clusters, while TVAE scattered broadly, reducing realism.
- **PCA (Figure 4.10)** – PCA analysis confirmed the same trend. TempusGAN generated points in regions where no other synthetic generator produced samples, showing that it is learning new variations that remain aligned with the real data structure. Other models produced overlapping clusters that added little diversity beyond what was already present.

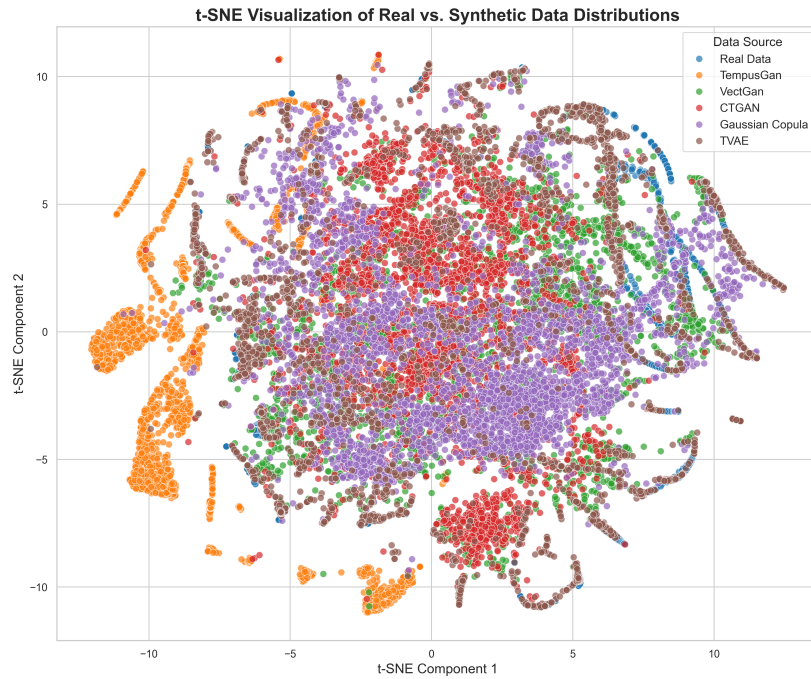


Figure 4.9: t-SNE visualisation of real and synthetic LAI samples. TempusGAN (orange) not only overlaps with real clusters (blue) but also explores new plausible regions absent in Normal and other generators, enriching underrepresented release behaviours.

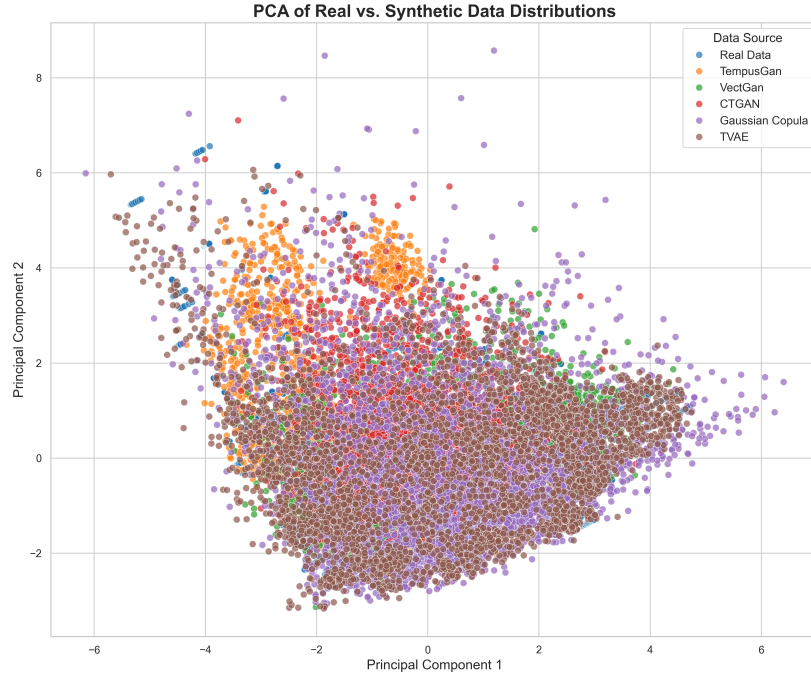


Figure 4.10: PCA projection of real and synthetic LAI samples. TempusGAN (orange) produces novel but coherent variations aligned with the real data structure, unlike CTGAN, VectGAN, and Gaussian Copula, which remain clustered or drift inconsistently.

4.3.4 Statistical Significance (LGBM, MAE)

Statistical significance testing was performed on the best-performing model, **LGBM**, using MAE of the test set. Paired t -tests confirmed TempusGAN’s superiority over all other synthesizers (Table 4.4).

Table 4.4: Paired t -test results for LGBM (LAI dataset, MAE).

Comparison	p-value	Significant ($p < 0.05$)
TempusGAN vs. VectGAN	0.031	Yes
TempusGAN vs. CTGAN	0.033	Yes
TempusGAN vs. Gaussian Copula	0.003	Yes
TempusGAN vs. TVAE	0.011	Yes
TempusGAN vs. Normal	0.032	Yes

Chapter 5

Discussion

Summary of Findings

This study asked a practical question: can a time-aware synthetic data generator improve predictive modelling for pharmaceutical release data when the datasets are small, noisy, and unevenly sampled? Across two different datasets, the results show that it can. TempusGAN consistently matched or outperformed both the non-augmented baseline and four widely used tabular synthesizers. On the polysaccharide coatings dataset, TempusGAN delivered the strongest XGBoost performance and clear improvements for kNN, with statistically significant advantages over CTGAN, TVAE, Gaussian Copula, VectGAN, and the baseline. On the LAI dataset, TempusGAN achieved the lowest MAE for LGBM and also led for Random Forest and NGB. That said, synthetic augmentation is not a guarantee of better performance: gains are contingent on the quality and coverage of the generated sequences, the augmentation ratio, and the choice of downstream learner. Poorly calibrated generators can introduce distribution shift, amplify experimental noise, or suffer mode collapse; excessive or mis-conditioned synthesis may even degrade accuracy. This is precisely why a domain-specific variational autoencoder is crucial: embedding pharmaceutical priors (e.g., monotonic and bounded release, condition-dependent kinetics, and realistic early/late-phase dynamics) constrains the latent space to biologically plausible regions, improving sample quality and reducing harmful drift. TempusGAN’s hybrid VAE-GAN was designed with these constraints, and as evidenced by the cross-dataset results, it overcame these pitfalls in practice, delivering reliable improvements.

Why Temporal Synthesis Helps

The key difference between TempusGAN and tabular synthesizers is what they choose to model. Tabular methods learn a joint distribution over features at a single point in time. In contrast, TempusGAN learns a distribution over complete time-indexed sequences, conditioned on formulation and physicochemical descriptors. This distinction matters for two main reasons:

Interpreting Results on the Coatings Dataset

The coatings dataset is extremely small and noisy, with only three timepoints per profile. Under these conditions, augmentation must add usable signal without amplifying noise. TempusGAN’s synthetic profiles, conditioned on Raman spectra and formulation descriptors, helped stabilize regressors. The strongest gains were observed in kNN and XGBoost.

For kNN, higher R^2 and lower MAE indicate that synthetic data populated sparse regions of feature space, improving neighborhood density and reducing variance in predictions. For XGBoost, TempusGAN achieved the lowest MAE with statistically significant improvements over all comparators, showing that tree-based models benefitted from more diverse but coherent training curves.

By contrast, VectGAN underperformed. Like CTGAN and TVAE, it is tabular and represents profiles as separate columns. With only three timepoints, this reduces the profile to three loosely related variables, allowing noise to dominate. Gaussian Copula and TVAE performed competitively in some folds, consistent with their strength in very low-dimensional tabular data, but neither matched the temporal consistency of TempusGAN.

The nested 5×5 cross-validation with 100 hyperparameter searches per inner loop ensured that these results were not due to random splits or tuning artefacts. This design makes the observed improvements more robust and reproducible.

Interpreting Results on the LAI Dataset

The LAI dataset is larger, more diverse, and contains richer descriptors. Non-augmented baselines already performed well, but TempusGAN still improved results. The clearest gains appeared for ensemble models such as LGBM, RF, and NGB. The lowest MAE with LGBM suggests that TempusGAN-generated sequences improved coverage of formulation–time interactions that were underrepresented in the real data. This is plausible because LAI profiles vary sharply with

polymer ratio, molecular weight, loading, and particle size. Synthetic diversity in these regions helps models generalize to new combinations.

Performance was less consistent for PLS, SVR, and kNN. Linear and kernel-based methods rely on smooth global structure and may not benefit from local diversity in the same way tree ensembles do. In datasets of moderate size, improvements depend on whether augmentation targets gaps in the data; when synthetic examples resemble already well-represented regions, these models gain little.

Consistency Across Downstream Models

The strongest and most consistent improvements were observed with tree-based ensembles: LGBM, RF, XGBoost, and NGB. These methods tolerate label noise, capture nonlinear interactions, and benefit directly from denser training distributions. SVR, PLS, and kNN were more variable, reflecting their sensitivity to feature scaling, kernel choice, and neighborhood geometry. This suggests that synthetic augmentation is most reliable when paired with flexible, robust learners such as tree ensembles.

Significance and Robustness

Paired t-tests confirmed that TempusGAN’s improvements were statistically significant for the best-performing downstream model in test set. Combined with the 5×5 nested cross-validation, this provides strong evidence that gains are real and reproducible rather than the result of random chance. The strict separation of training and test folds during synthesis further ensured that no information leakage biased results.

What Augmentation Achieves

Augmentation worked through two mechanisms:

- **Densification:** Filling gaps in the joint space of formulation descriptors and trajectory shapes, reducing the need for unstable extrapolation.
- **Regularisation:** Providing coherent near-neighbors, which smooths predictive mappings along biologically meaningful directions.

TempusGAN’s temporal modelling meant this smoothing followed the natural structure of release curves, unlike tabular methods that blur disconnected columns.

Limitations

- **Dependence on preprocessing.** Resampling, baseline correction, and normalization influence what the model learns. Artefacts in preprocessing can be reproduced by the generator.
- **Training cost.** TempusGAN is more complex than tabular models and requires longer training, though still far cheaper than collecting new laboratory data.

Generalisation Across Datasets

The two datasets illustrate different regimes. For coatings, TempusGAN rescued performance in an extremely small, noisy dataset by aggressive augmentation. For LAI, it enriched diversity in a larger dataset where baselines were already strong. This shows TempusGAN’s flexibility: it can serve both as a stabiliser in very small studies and as a performance enhancer in moderate datasets with underrepresented combinations.

Practical Guidance

Several practical lessons emerge:

- Match augmentation volume to dataset size: aggressive ratios help in small datasets, moderate ratios suffice for larger ones.
- Use tree-based models as a default downstream choice, since they benefit most reliably.
- Keep synthetic generation strictly inside training folds to prevent leakage.
- Report not only mean performance but also variance reduction, since reproducibility matters in real pharmaceutical development.

Implications for Pharmaceutical R&D

Generating real release data is expensive and time-consuming. If synthetic augmentation reduces the number of experiments required to reach a robust predictive model, it can save both resources and time. This is valuable in early-stage screening of formulations and exploration of new drug–polymer combinations, where

the design space is large but data are sparse. Synthetic data can also facilitate safer data sharing by providing realistic but non-identical trajectories, easing collaboration across institutions while protecting sensitive experimental datasets.

Chapter 6

Conclusion

This work set out to determine whether a time-aware generative model could improve predictive modelling for pharmaceutical release data that are often small, noisy, and irregular. Across two very different datasets, the answer was clear: TempusGAN consistently matched or outperformed both traditional baselines and existing synthetic data generators. On the polysaccharide coatings dataset, where only three release timepoints were available, TempusGAN produced coherent synthetic profiles that stabilised regressors and yielded significant gains, especially for XGBoost and kNN. On the larger LAI dataset, TempusGAN again improved performance, particularly for ensemble methods such as LGBM, RF, and NGB, by enriching the diversity of training examples. These findings confirm the central hypothesis: when modelling time-dependent release processes, generating full trajectories is more effective than tabular augmentation.

The study also demonstrated that these improvements were robust and reproducible. The use of 5×5 nested cross-validation with extensive hyperparameter searches ensured fairness across models, while strict separation of training and test folds prevented leakage. The benefits observed were therefore due to the quality of TempusGAN’s temporal modelling rather than artefacts of evaluation. More broadly, the results highlight the value of synthetic augmentation in pharmaceutical research: TempusGAN offers a way to make better use of limited experimental data, reduce variance in small studies, and improve generalisation in larger datasets. By producing biologically plausible trajectories that reflect the underlying release mechanisms, TempusGAN represents a step towards more data-efficient and reliable predictive modelling in drug development.

Finally, TempusGAN has potential for privacy-preserving data sharing. Pharmaceutical datasets are often difficult to share due to intellectual property and regulatory concerns. If TempusGAN can generate fully synthetic datasets that maintain the same statistical and mechanistic properties as the originals, other researchers could use these synthetic versions to train and benchmark predictive models with-

out accessing sensitive raw data. This would make collaboration easier across institutions, promote reproducibility, and enable the development of new modelling approaches even when real data cannot leave a laboratory or company.

Chapter 7

Future Work

We will extend TempusGAN to two additional datasets to test generality beyond coatings and LAIs: (1) a microneedle patch dataset for in-vitro drug permeation prediction, and (2) a dataset on drug-loaded PLGA microparticles [1], which spans diverse formulation parameters and release behaviors. These studies will probe whether TempusGAN’s gains transfer to different release mechanisms and time scales, and whether augmentation still helps when base models are already strong. We will also harden validation. Future validation will include explicit checks on synthetic data quality. Alongside downstream metrics, we will add distributional diagnostics (coverage, dynamic time warping to nearest real profiles, and failure-mode audits by formulation subgroup) to ensure that synthetic data improves models without introducing artifacts.

Interpretability and knowledge extraction are a priority. We will run descriptor-wise counterfactuals (change one variable at a time) and fit simple surrogate models on TempusGAN outputs to map how polymer MW, particle size, loading, and LA/GA ratio shift onset, slope, and plateau. Coupled with attribution on the decoder, this will translate generated trajectories into practical design rules that formulators can act on.

Finally, we will develop a privacy-preserving release pipeline: generate fully synthetic cohorts that match the statistical and mechanistic structure of the originals, certify them with the above validation suite, and distribute them for benchmarking. The goal is to enable external teams to train and compare predictive models without accessing sensitive raw data, improving collaboration and reproducibility while protecting patient- or product-level confidentiality.

Code Availability

The full implementation of **TempusGAN**, including core model components, training and evaluation utilities, and example configuration files, is openly available at: <https://github.com/suprix17/Tempus-GAN>.

Bibliography

- [1] Zeqing Bao et al. “Polymer microparticles in an evolving drug delivery landscape: challenges and the role of machine learning”. In: *International Journal of Pharmaceutics* — (2025). PMID: 40578461, p. 125906. DOI: [10.1016/j.ijpharm.2025.125906](https://doi.org/10.1016/j.ijpharm.2025.125906). URL: <https://doi.org/10.1016/j.ijpharm.2025.125906>.
- [2] Sandra Klein. “Predicting Food Effects on Drug Release from Extended-Release Oral Dosage Forms Containing a Narrow Therapeutic Index Drug”. In: *Dissolution Technologies* 16.3 (2009), pp. 28–35. DOI: [10.14227/DT160309P28](https://doi.org/10.14227/DT160309P28).
- [3] Chiun Zhen. “Pharmacokinetic Modeling of Controlled Release Drug Delivery Systems”. In: *Der Pharmacia Lettre* 16.11 (2024), pp. 11–12.
- [4] Youssef Abdalla et al. “Machine learning of Raman spectra predicts drug release from polysaccharide coatings for targeted colonic delivery”. In: *Journal of Controlled Release* 374 (2024), pp. 103–111. DOI: [10.1016/j.jconrel.2024.08.010](https://doi.org/10.1016/j.jconrel.2024.08.010). URL: <https://doi.org/10.1016/j.jconrel.2024.08.010>.
- [5] Pauric Bannigan et al. “Machine learning models to accelerate the design of polymeric long-acting injectables”. In: *Nature Communications* 14.1 (2023), p. 35. DOI: [10.1038/s41467-022-35343-w](https://doi.org/10.1038/s41467-022-35343-w). URL: <https://doi.org/10.1038/s41467-022-35343-w>.
- [6] Kirsten Zantvoort, Barbara Nacke, and Dennis Görlich. “Estimation of minimal data set sizes for machine learning predictions in digital mental health interventions”. In: *npj Digital Medicine* 7.361 (2024), pp. 1–10. DOI: [10.1038/s41746-024-00961-7](https://doi.org/10.1038/s41746-024-00961-7). URL: <https://www.nature.com/articles/s41746-024-01360-w>.
- [7] PhRMA Foundation. *Drug Delivery Research Area*. 2023. URL: <https://www.phrmafoundation.org/research-areas/drug-delivery/>.

- [8] Northeastern University Bouve College of Health Sciences. *Why Study Pharmaceutical Sciences?* 2024. URL: <https://bouve.northeastern.edu/news/why-study-pharmaceutical-sciences>.
- [9] Marzieh Fadaee, Arianna Bisazza, and Christof Monz. “Data Augmentation for Low-Resource Neural Machine Translation”. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2017, pp. 567–573. DOI: [10.18653/v1/P17-2090](https://doi.org/10.18653/v1/P17-2090). URL: <https://aclanthology.org/P17-2090/>.
- [10] Jason Wei and Kai Zou. “EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks”. In: *EMNLP*. 2023. DOI: [10.48550/arXiv.1901.11196](https://doi.org/10.48550/arXiv.1901.11196). URL: <https://arxiv.org/abs/1901.11196>.
- [11] RJ Chen et al. “Synthetic data in machine learning for medicine and healthcare”. In: *Nature Biomedical Engineering* 5 (2021), pp. 493–497. DOI: [10.1038/s41551-021-00751-8](https://doi.org/10.1038/s41551-021-00751-8). URL: <https://doi.org/10.1038/s41551-021-00751-8>.
- [12] K. Hartmann, R. Tibor Schirrmeister, and T. Ball. “EEG-GAN: Generative adversarial networks for electroencephalographic (EEG) brain signals”. In: *arXiv preprint*. 2018. DOI: [10.48550/arXiv.1806.01875](https://doi.org/10.48550/arXiv.1806.01875). URL: <https://arxiv.org/abs/1806.01875>.
- [13] Yifan Zhao Zhishui You. “A Review on Deep Learning-Based Synthetic EEG Generation Methods”. In: *MDPI* 15 (2021), p. 14. DOI: [10.3389/fncom.2021.649619](https://doi.org/10.3389/fncom.2021.649619). URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12116193/>.
- [14] Zhiguang Wang, Weixun Yan, and Tim Oates. “Time Series Classification from Scratch with Deep Neural Networks: A Strong Baseline”. In: *International Joint Conference on Neural Networks (IJCNN)* (2020). DOI: [10.1109/IJCNN48605.2020.9207326](https://doi.org/10.1109/IJCNN48605.2020.9207326). URL: <https://ieeexplore.ieee.org/document/7966039>.
- [15] Alex Zhavoronkov et al. “Deep learning enables rapid identification of potent DDR1 kinase inhibitors”. In: *Nature Biotechnology* 37 (2019), pp. 1038–1040. DOI: [10.1038/s41587-019-0224-x](https://doi.org/10.1038/s41587-019-0224-x). URL: <https://doi.org/10.1038/s41587-019-0224-x>.
- [16] Daniil Polykovskiy and et al. “Molecular Sets (MOSES): A Benchmarking Platform for Molecular Generation Models”. In: *Frontiers in Pharmacology* 11 (2020), p. 565644. DOI: [10.3389/fphar.2020.565644](https://doi.org/10.3389/fphar.2020.565644). URL: <https://doi.org/10.3389/fphar.2020.565644>.

- [17] Youssef Abdalla et al. “VECT-GAN: A variationally encoded generative model for overcoming data scarcity in pharmaceutical science”. In: *arXiv preprint* arXiv:2501.08995 (2025). DOI: [10 . 48550 / arXiv . 2501 . 08995](https://doi.org/10.48550/arXiv.2501.08995). URL: <https://arxiv.org/abs/2501.08995>.
- [18] SDV developers. *SDV: Synthetic Data Vault*. 2023. URL: <https://github.com/sdv-dev/SDV>.
- [19] Lei Xu et al. “Modeling tabular data using conditional GAN”. In: *NeurIPS*. 2019. URL: <https://proceedings.neurips.cc/paper/2019/hash/254ed7d2de3b23ab10936522dd547b78-Abstract.html>.
- [20] SDV developers. *TVAESynthesizer*. SDV documentation. 2024. URL: <https://docs.sdv.dev/sdv/single-table-data/modeling/synthesizers/tvaesynthesizer>.
- [21] Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. “Time-series generative adversarial networks”. In: *NeurIPS*. 2019. URL: https://proceedings.neurips.cc/paper_files/paper/2019/file/c9efe5f26cd17ba6216bbe2a7d26d490-Paper.pdf.